데이터와 LLM을 실제 서비스로 연결하는 AI 엔지니어

RAG·Backend·Data Pipeline 전반의 설계와 구현

KEY SKILLS

Python TypeScript FastAPI Next.js LLM RAG PostgreSQL Docker

SOFT SKILLS

프로젝트 리딩 문제 정의·해결 부서 간 협업

Contents

PROJECTS

MS Data School | 프로젝트 3건 | 공통 역할: AI 서비스·데이터 파이프라인·팀 리딩

프로젝트	역할	인원	기간	핵심기술	주요성과
공공조달 입찰 요건 분석·대응 자동화 플랫폼	데이터·LLM 엔지니어	6인	2026.05.14-05.29	Azure OpenAI, AI Search, Azure Functions	전체 모집단 대비 후속 처리 대상 **95.1% 축소**(15,894→779건), Azure 리소스 **8개** 배포 자동화
오프보딩 이후 잔여 접근권한 탐지·조치 지원 플랫폼	데이터 엔지니어	5인	2026.06.22-07.09	Vector Search, LLM API, PySpark	7개 시스템·16개 객체를 **4개 엔티티**로 통합, 주입한 오류 **37건 전부 탐지**
취약점 재평가·조치 자동화 플랫폼	데이터·풀스택 엔지니어	6인	2026.08.03-09.04	Microsoft Fabric, FastAPI, PostgreSQL, RAG	취약점 알림 **406→66→10건** 압축, 위험도 급등 취약점 **30건(8.7%)** 검증

CAREER

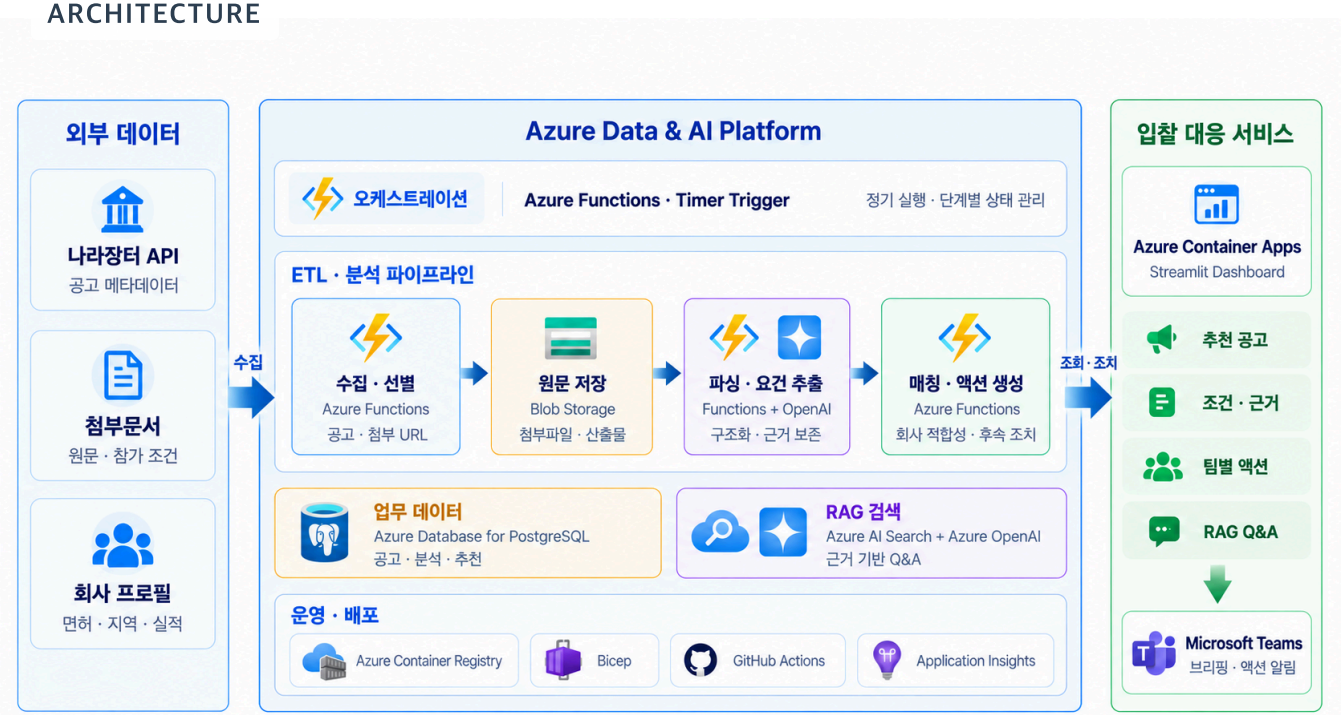
경력	역할	인원	기간	핵심기술	주요성과
이큐비알홀딩스	블록체인 연구원	4인	2021.01-2022.12	Go, MongoDB, AWS, Docker	**TPS 3,000→4,500** 향상, 백필 전송량 **60~70% 절감**

PROJECT 1

공공조달 입찰 요건 분석·대응 자동화 플랫폼

2026.05.14-2026.05.29 | 데이터·LLM 엔지니어 | Azure OpenAI | AI Search | Azure Functions | RAG

ARCHITECTURE



프로젝트 배경

- 나라장터 공고는 확인할 정보가 많고, 중요한 참가 요건은 첨부문서 안에 있어 공고만 보고는 즉시 판단하기 어려움
- 요건을 추출해도 실제 대응은 담당 부서별로 나눠 확인해야 해 검토가 분산됨

역할

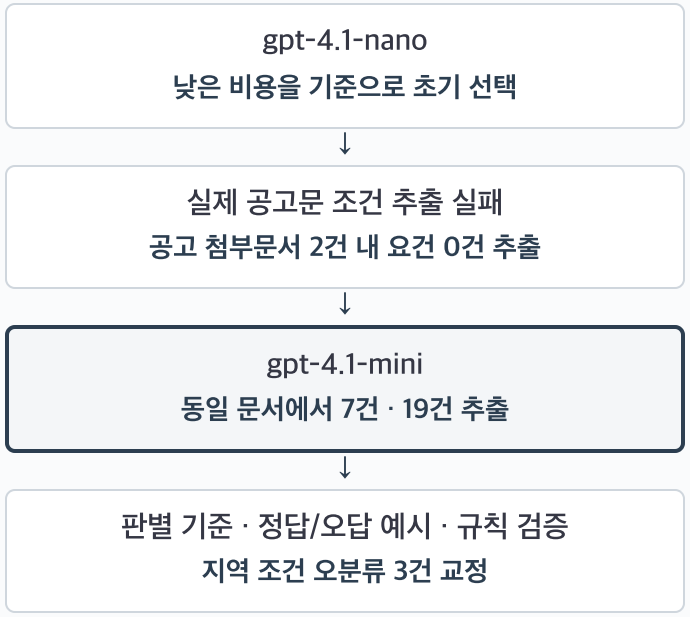
- 나라장터 공고·첨부문서 수집부터 요건 구조화·매칭·액션 생성까지 데이터 파이프라인 구현
- LLM 기반 요건 추출, 신뢰성 있는 RAG 질의응답 구현
- Bicep·GitHub Actions 기반 인프라 배포 자동화

핵심 결과

- 전체 모집단 대비 후속 처리 대상 **95.1% 축소**(15,894건→779건)
- LLM·규칙 기반 검증으로 입찰 조건 오분류 완화
- **8개 Azure 리소스** 구성·배포 표준화

PROJECT 1 · 핵심 챌린지

실제 공고문 기반 모델 평가와 추출 품질 개선



문제

비용을 기준으로 선택한 초기 모델이 명시적인 참가 조건이 있는 실제 공고문 2건에서 조건을 한 건도 추출하지 못해, 비용과 추출 품질을 함께 고려한 모델 재선정 필요

해결

동일한 실제 문서로 후보 모델을 비교하고, 비용 증가보다 필수조건 누락의 영향을 우선해 모델 선택

01 추출 품질과 비용 비교

실제 첨부파일 6건으로 nano와 mini를 비교. mini의 입력 단가는 4배였지만 샘플 처리비 약 0.01달러, 월 수백 건 기준 약 1~2달러로 추정돼 필수조건 추출을 우선

02 판별 기준과 정답·오답 예시 추가

업체 소재지를 제한하는지를 판별 기준으로 명시하고 정답·오답 예시를 추가. 정규식·키워드 규칙으로 LLM 결과를 재검증해 일관성 보완

결과

- 동일한 실제 공고 2건의 조건 항목 추출 **0→7건, 0→19건**
- 빈 문서 0건 추출 유지, 지역 조건 오분류 **8건 중 3건 교정**

PROJECT 1 · 핵심 챌린지

AI 서비스의 배포 자동화와 관측 환경 구축



문제

LLM·검색·API 기능이 8개 Azure 리소스에 걸쳐 있어 수동 구성 시 모델 배포명과 엔드포인트 설정이 달라지고, 동일한 서비스 환경을 재현하기 어려움

해결

AI 기능을 구성하는 리소스와 연결 설정을 코드화하고, 애플리케이션과 동일한 CI/CD 흐름으로 관리

01 AI 서비스 실행 환경 코드화

Bicep으로 Azure 리소스 8개의 구성과 연결 설정을 정의하고 GitHub Actions에서 검증·배포해 동일한 절차로 환경 구성

02 로그·실패·알림 연결

Application Insights로 API와 워크플로 실행 로그를 확인하고, 오류 발생 시 Teams 알림으로 연결해 실패 지점을 추적할 수 있는 관측 환경 구성

결과

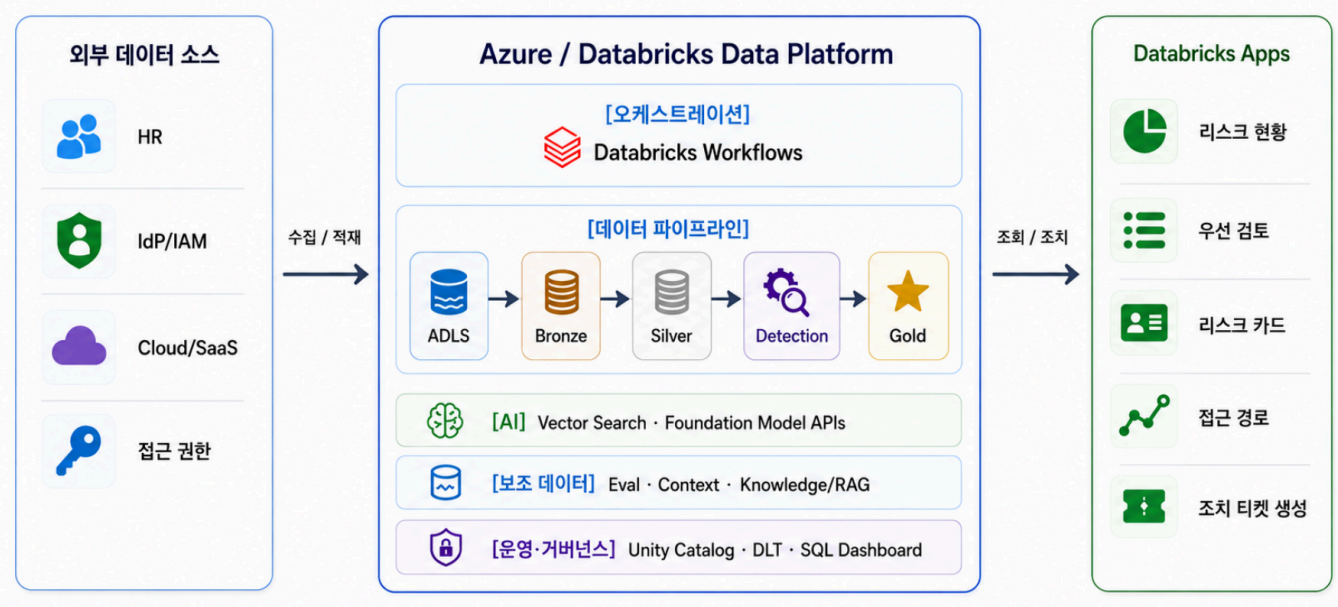
- **Azure 리소스 8개**의 구성·연결 설정을 코드로 표준화
- 동일한 IaC 정의를 통한 반복 배포와 서비스 환경 재구성 가능
- 로그와 오류 알림을 통해 배포 이후의 실패 원인 확인 가능

PROJECT 2

오프보딩 이후 잔여 접근권한 탐지·조치 지원 플랫폼

2026.06.22-2026.07.09 | 데이터 엔지니어 | Vector Search | LLM API | PySpark | Databricks

ARCHITECTURE



프로젝트 배경

- 퇴사자 계정을 비활성화해도 자동화 계정·인증 정보가 별도 생명주기로 남아 간접 접근 경로가 생길 수 있음
- 시스템마다 계정·권한 표현 방식이 달라 이런 경로를 한 번에 확인하기 어려움

역할

- 다중 시스템 접근 데이터의 공통 그래프 모델·ELT 설계
- 정책·조치 문서의 RAG 근거 데이터 계약과 적재 파이프라인 구현
- 근거 기반 응답·출처 인용·담당자 승인 원칙과 검색 인터페이스 설계 참여

핵심 결과

- **7개 시스템·16개 데이터 객체**를 4개 공통 엔티티로 통합
- 주입 오류 **37건 전부 탐지** 및 하위 레이어 전파 차단
- 출처·변경 이력을 포함한 초기 RAG 근거 코퍼스 **9개 청크** 구축

PROJECT 2 · 핵심 챌린지

신뢰 가능한 RAG를 위한 근거 코퍼스 설계



문제

근거 선택: 보안 조치에 필요한 정책·절차·API 근거를 리스크와 시스템에 맞게 구분해 검색하고, 원문까지 확인할 수 있는 구조 필요

데이터 경계: 인증정보 원문이 코퍼스와 LLM 입력으로 전달될 위험

해결 과정

문서 유사도에만 의존하지 않도록 근거의 적용 범위와 출처를 메타데이터로 명시

01 근거 문서 큐레이션·정규화

문서의 역할이 검색 결과에서 뒤섞이지 않도록 필요한 근거를 직접 선별하고, 적용할 리스크와 시스템·조치 유형을 부여해 공통 구조로 변환

02 출처·민감정보 검증

검색 결과를 원문까지 확인할 수 있도록 근거 식별자와 출처·변경 이력을 저장하고, 인증정보 원문과 중복 데이터는 적재 전에 차단

결과

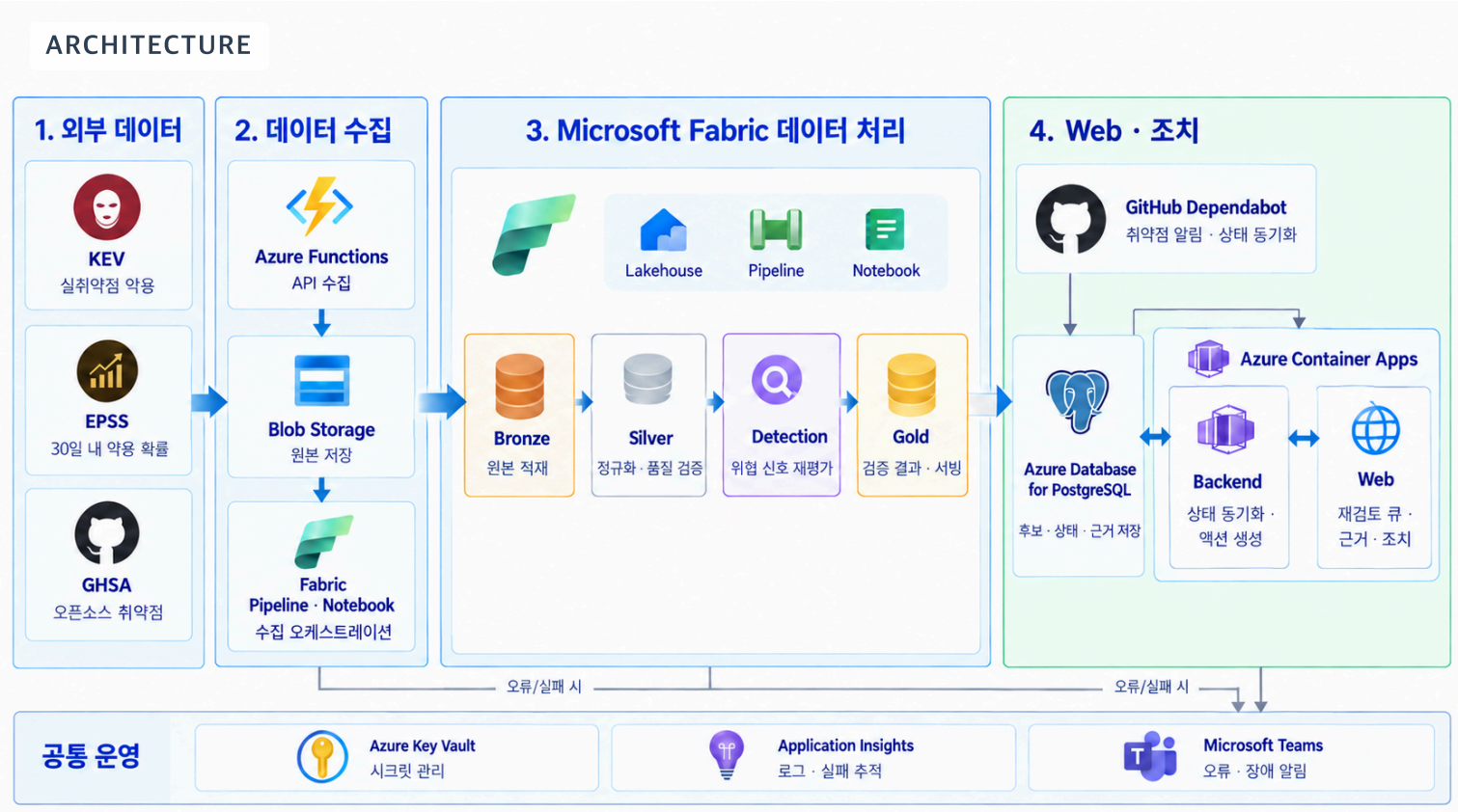
- 데이터 계약·DDL·적재 파이프라인과 초기 **9개 근거 청크** 구현
- 조건 기반 Vector Search와 답변 출처 추적 기반 확보
- “퇴사자 소유 앱의 활성 인증정보를 어떻게 조치할 것인가?” 등 **평가 시나리오 7건**과 정답 근거 설계

PROJECT 3

취약점 재평가·조치 자동화 플랫폼

2026.08.03-2026.09.04 | 데이터·풀스택 엔지니어 | Microsoft Fabric | FastAPI | PostgreSQL | RAG

ARCHITECTURE



프로젝트 배경

- 취약점의 악용 가능성은 공개 이후에도 계속 변할 수 있음
- Dependabot 알림이 저장소·의존성 파일별로 분산돼 위험 분석 결과를 조치 단위로 연결하기 어려움

역할

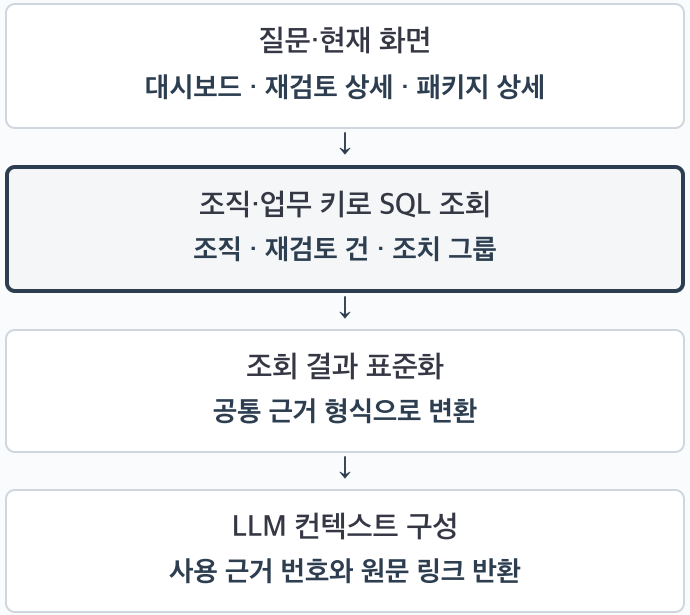
- 소스별 갱신 특성에 맞춘 수집 전략 설계
- Fabric 취약점 분석 결과와 PostgreSQL 취약점 알림을 연결하는 동기화 구조 설계
- PostgreSQL 운영 DB와 FastAPI·Next.js 기반 재평가·조치 서비스 구현

핵심 결과

- EPSS 일별 위험도 이력 누적 **1.6억+건** 수집·보존
- 전체 취약점 **알림 406건 → 위험변화 매칭 66건 → 검토·조치 단위 10건**으로 단계적 압축

PROJECT 3 · 핵심 챌린지

현재 화면과 조직에 맞는 근거로 LLM 컨텍스트 구성



문제

재검토 건과 패키지 조치에 필요한 운영 데이터와 분석 문서가 여러 테이블에 분산돼 있어, 현재 화면과 조직에 맞는 근거만 조합해 LLM에 전달할 필요

해결 과정

정형 데이터는 SQL로 조회하고, 비정형 문서에만 벡터 검색을 적용하도록 검색 역할 분리

01 화면별 SQL 조회와 조직 검증

질문의 업무 맥락에 따라 SQL 조회 범위를 조직과 검토 대상으로 제한. LLM 호출 전 조직 범위를 다시 확인해 불일치 시 응답 생성 차단

02 분산된 근거의 공통 구조화

조회한 운영 데이터와 분석 문서를 공통 근거 형식으로 변환하고, 답변에 실제 사용된 근거 번호와 원문 링크를 함께 반환

결과

- 대시보드·재검토 건·패키지의 **3단계 화면별 컨텍스트** 구성
- 다른 조직의 근거가 포함되면 LLM 호출 전 응답 생성 차단
- 답변에 실제 사용된 근거만 식별자·원문 링크와 함께 반환

CAREER

프로덕션 데이터 흐름과 운영 환경 개선

(주)이큐비알홀딩스 | 2021.01-2022.12 | 블록체인 연구원

데이터 서빙 구조 개선

- 합의 노드에서 원장 복제 노드를 분리하고 CQRS 기반 RDB 서빙 레이어 구축
- 원장 데이터를 실시간 스트림으로 적재하고 순차 정수 인덱스로 조회 경로 개선
- gzip 압축 백필과 이벤트 재생 기반 상태 재구성 구현

코어 전환과 데이터 호환

- PHP 기반 레거시 시스템을 검증된 Go 오픈소스 클라이언트 기반으로 전환
- 기존 데이터를 신규 클라이언트에서 처리할 수 있는 호환 구조 구현
- 합의 관련 데이터·메시징 인터페이스를 자사 네트워크 요구사항에 맞게 확장

개발·운영 환경 자동화

- Docker Compose로 다중 P2P 노드의 빌드·구성·피어링 자동화
- 노드 연결·합의 상태를 확인하는 시각화 도구 개발
- Grafana 기반 통신 지연·DB 사용량 모니터링 환경 구축

60~70% 절감

약 100만 블록·수백 GB 원장 백필 전송량

TPS 3,000→4,500

Go 기반 코어 전환 및 인터페이스 확장

50%+ 단축

개발·테스트 환경 구성 시간